Preferential Web Crawler for Image Segmentation Content on GitHub: Implementation, Architecture, and Evaluation

Tilen Lampret & Pika Križnar & Grega Radež
Faculty of Computer and Information Science, Ljubljana
email: tl2363@student.uni-lj.si

ABSTRACT
This report describes the design, implementation, and evaluation of a preferential web crawler targeting image segmentation content on GitHub. The crawler is built around a two-stage producer–consumer architecture in which a persistent PostgreSQL frontier orders discovered URLs by relevance score, and a domain-scoped pump coordinates fetch scheduling to satisfy per-host politeness constraints. Seed selection followed a deliberate two-step process: an initial set of well-connected repositories was used to surface the most relevant topic pages, which were then hand-curated and added as secondary seeds for the main crawl. The production crawl collected 5,000 HTML pages and 197,401 links from `github.com`, with no duplicate pages detected. The resulting dataset is thematically coherent and well-suited for downstream use in retrieval-augmented generation systems focused on image segmentation.

Key words: preferential crawling, image segmentation, web frontier scheduling

1 ASSIGNMENT INTENT AND HOW WE ADDRESSED IT

The assignment specifies a standalone preferential crawler structured around five core components: fetch and render, link and image extraction, duplicate handling, frontier management, and database persistence. Beyond this decomposition, the requirements mandate parallel workers with a configurable count, robots.txt compliance including crawl-delay and sitemap awareness, a politeness policy enforcing at least one request per host every five seconds unless robots demands a longer delay, canonical URL storage, content-level duplicate detection with URL-level deduplication handled without re-fetching, link extraction from `href` attributes and relevant `onclick` patterns, image metadata collection from `img[src]`, HTML and PDF page bodies stored as downloadable content, non-HTML binary types recorded without storing their bytes, a fixed User-Agent string of `fri-wier-IEPS-TOLPA`, and avoidance of turnkey crawler frameworks listed as forbidden.

The crawler targets `github.com` with a focus on image segmentation, covering its topic pages, representative repositories, and a bounded exploration of related content. This choice is consistent with the assignment's suggested GitHub subset pattern: the crawler collects metadata and HTML from README-style views and topic pages rather than cloning repositories or exhaustively traversing directory trees, in accordance with the ethical and scope guidance outlined in the assignment.

Regarding the optional bonus, the assignment describes LSH-based Jaccard near-duplicate detection as an extension. The delivered implementation follows the required content deduplication approach of exact-match identity via SHA-256 content hashing with duplicate handling at the database level, and does not claim the LSH bonus.

2 ARCHITECTURE

The crawler is built around a two-stage producer–consumer architecture. Stage A is responsible for ingestion: transforming every newly discovered URL into a scored, deduplicated, robots-checked frontier entry. Stage B is responsible for execution: claiming entries from the frontier, fetching and parsing pages, persisting results, and feeding newly discovered links back through Stage A. PostgreSQL serves as the authoritative store for both the frontier and the crawl graph, making all state persistent and consistent across workers without relying on in-process structures that would be lost on failure. The system is designed to run as a single

process per database instance, since operating multiple concurrent JVMs against the same database would undermine both the per-domain rate limiting and the robots single-flight guarantees that the architecture depends on.

Stage A defines a strict ordered pipeline applied to every URL that emerges from parsing. The URL is first canonicalized through scheme normalization, fragment stripping, and parameter filtering, so that only one canonical form ever enters the system. It is then evaluated against the robots.txt rules for its host, which are fetched and cached in a shared `RobotsTxtCache` accessible to both stages. Robots fetches themselves participate in the same per-domain rate limiter as content fetches, so they do not bypass the politeness contract. If the URL is disallowed, it is dropped at this point rather than deferred. URL-level deduplication is handled next through an `insertFrontierIfAbsent`-style upsert, where the database is authoritative and the upsert returns the existing `page_id` on conflict so that no duplicate frontier row is ever created. A relevance score is then computed from configurable keyword weights applied to the URL path, anchor text, and surrounding textual context extracted during parsing. This score is stored with the frontier row so that the frontier can be ordered by `relevance_score DESC` with an age-based tie-break. If the frontier has reached its size cap, the incoming URL replaces the lowest-scoring existing `FRONTIER` row only when its score is strictly higher, ensuring that preferential ordering is preserved under a finite budget. Finally, regardless of whether the URL was new or already known, a `link` edge is inserted into the graph to record the relationship between the source page and the discovered URL, preserving graph completeness even for URLs that produced no new frontier row.

Seed URLs follow a special bootstrap path at startup. They are canonicalized and assigned a fixed high-floor score from configuration, a `site` row is created for each seed domain, and seeds are inserted into the frontier idempotently. Seeds are not HTTP-fetched during bootstrap. Robots evaluation for a seed domain is deferred to Stage B, when a worker is about to fetch it for the first time.

Stage B begins when the domain-scoped frontier pump claims a frontier row. The claim is atomic, transitioning the row from `FRONTIER` to `PROCESSING` with a lease timestamp, so no two workers can act on the same URL simultaneously. The pump operates per crawl domain rather than globally. Instead of having all workers compete for the single highest-scored URL in the entire frontier, the pump identifies the next domain that is not currently rate-limited and has no active worker lease, then claims the best-scored eligible URL for that domain. This design is necessary because the crawler targets `github.com` almost exclusively, meaning a global claim strategy would cause every worker to converge on the same host, stall on the politeness gate, and generate excessive lease churn. By scheduling at the domain level, the pump keeps the worker pool occupied without violating the politeness contract, and allows any other domains present to progress in parallel.

Once a URL is claimed, the worker passes it through the politeness gate via `PolitenessGate.tryAcquire`. If the gate cannot be acquired immediately, for example because a prior request to the same host was too recent, the lease is rescheduled to `FRONTIER` with a future `next_attempt_at` timestamp and the domain is woken later, so no worker thread busy-waits and no rate-limit token is consumed unnecessarily. When the gate is acquired, robots.txt rules are re-evaluated immediately before the fetch. A `DISALLOWED` result terminates the URL as an error, while a `TEMPORARY_DENY` result reschedules it. Only URLs that pass both gates proceed to `fetcher.fetch()`, which follows redirects hop-by-hop, applying robots and rate-limit checks at each hop. The first hop of the fetch does not consume a second politeness token because the pump already acquired one on the worker's behalf before submission, and inner redirect hops follow the fetcher's own rules.

After a successful fetch, `parser.parse()` produces the page content artifacts and the set of discovered URLs with their source page identifiers attached. All outcomes are then committed in a single `SERIALIZABLE` transaction via `persistFetchOutcomeWithLinks`: the page row is updated to its final type of `HTML`, `BINARY`, or `DUPLICATE`, retry metadata is recorded where applicable, and Stage A ingestion is executed in batch for every discovered URL. This closes the loop, as new links produced by parsing are admitted to the frontier within the same transaction as the fetch outcome that produced them. On retryable failures the lease is rescheduled, and on non-retryable failures the page is marked as an error. Stale `PROCESSING` leases whose timeout has elapsed are reclaimed to `FRONTIER` by a background recovery process, ensuring that a crashed or hung worker does not permanently remove a URL from the crawl.

3 SEED SELECTION AND CRAWL CONFIGURATION

Seed selection was carried out in two deliberate steps rather than through a single static list. In the first step, a small set of well-known and highly connected repositories was chosen as initial seeds. These repositories were selected specifically because of the large number of outgoing links they contain, which allowed the crawler to rapidly widen the set of available frontier candidates from the very first fetch. During this phase, the relevance scoring was configured to amplify scores based on keyword occurrence density: if a primary term such as *segmentation* appeared many times in the surrounding context of a discovered link, the base score

Table 1. Seed URLs used for the production crawl, combining an initial set of well-connected repositories with topic pages identified during the exploratory seeding phase.

Type	URL
Repository	`https://github.com/facebookresearch/segment-anything`
Repository	`https://github.com/open-mmlab/mmsegmentation`
Repository	`https://github.com/jfzhang95/pytorch-deeplab-xception`
Repository	`https://github.com/milesial/Pytorch-UNet`
Repository	`https://github.com/TexasInstruments/edgeai-modelzoo`
Repository	`https://github.com/onnx/models`
Topic	`https://github.com/topics/image-segmentation`
Topic	`https://github.com/topics/semantic-segmentation`
Topic	`https://github.com/topics/medical-image-segmentation`
Topic	`https://github.com/topics/medical-image-processing`
Topic	`https://github.com/topics/unet`
Topic	`https://github.com/topics/unet-pytorch`
Topic	`https://github.com/topics/unet-image-segmentation`
Topic	`https://github.com/topics/dpt`
Topic	`https://github.com/topics/imagenet`
Topic	`https://github.com/topics/image-processing`
Topic	`https://github.com/topics/lane-detection`
Topic	`https://github.com/topics/video-segmentation`
Topic	`https://github.com/topics/computer-vision`
Topic	`https://github.com/topics/segmentation-models`
Topic	`https://github.com/topics/satellite-imagery`
Topic	`https://github.com/topics/panoptic-segmentation`
Topic	`https://github.com/topics/fpn`
Topic	`https://github.com/topics/food-classification`

was multiplied by the number of occurrences. This caused the crawler to converge quickly on GitHub topic pages, which aggregated the highest concentrations of relevant terminology and therefore ranked consistently above other page types.

After allowing the crawler to surface the thirty highest-scoring topic pages in this manner, the team manually reviewed the results and selected the most relevant topic pages based on the quality and thematic focus of the repositories linked from each one. This hand-curation step produced the final seed list shown in Table 1, which combines the original repository seeds with the curated topic page seeds. By using this two-step process, the seed list reflects both broad connectivity and domain-specific relevance, providing a much stronger starting point than either approach would have achieved alone.

Following the completion of seed selection, topic pages were explicitly excluded from the frontier for the main production crawl. Although topic pages had been valuable during the seeding phase, their structure made them systematically over-scored during normal crawling: because topic pages aggregate large numbers of relevant keywords in their tag clouds and page metadata, they consistently outscored repository pages regardless of the keyword tuning applied. Allowing topic pages to remain in the frontier would have caused the crawler to repeatedly fetch index-style pages with limited textual content rather than the repository pages that carry substantive information. Since the intended downstream application is a retrieval-augmented generation system, repository pages with rich README content, code descriptions, and documentation are considerably more valuable than topic index pages, and the exclusion was therefore both necessary and well-motivated.

4 RUNTIME PARAMETERS AND CONFIGURATION

The crawler is configured entirely through a properties file, with any key overridable via environment variable by converting `crawler.*` keys to `CRAWLER_*` with dots replaced by underscores. For example, `crawler.db.url` becomes `CRAWLER_DB_URL` and `crawler.seedUrls` becomes `CRAWLER_SEEDURLS`. The system requires JDK 21 for both the Gradle build and the runtime, as newer JDK versions may break the build. PostgreSQL must be available before the crawler starts, with the schema applied from `pa1/db/crawldb.sql`. The expected schema version is 7, and the value of `crawler.db.expectedSchemaVersion` must match the applied schema exactly. Chrome or Chromium must also be available for Selenium when HTTP-only fetching is insufficient for a given page.

Database connectivity is specified through `crawler.db.url`, `crawler.db.user`, and `crawler.db.password`, which together form the JDBC connection to PostgreSQL. The connection pool size is controlled by `crawler.db.poolSize`

and should be kept consistent with the configured concurrency level and the headless browser session limit, as mismatches can exhaust database connections under load.

The crawl budget is governed by two parameters. `crawler.budget.maxTotalPages` sets a hard cap on the number of distinct page rows that will be inserted, and `crawler.budget.maxFrontierRows` sets an optional watermark on frontier size beyond which score-based eviction applies. Seeds are specified as a comma-separated list of HTTPS URLs in `crawler.seedUrls` and are only consumed when the database is empty at startup, making it safe to restart a crawl without re-processing them. The crawl scope is fixed by `crawler.crawlScope`, which when set to `GITHUB` restricts in-scope hosts to `github.com` and `www.github.com` while excluding all other subdomains. Topic page discovery can be suppressed independently via `crawler.discovery.blockGithubTopicsPaths`, which prevents any `/topics` URL from being enqueued during the main crawl while still allowing topic pages to appear as seeds.

Relevance scoring is driven by a keyword configuration file whose path is set in `crawler.scoring.keywordConfig`. When running via `./gradlew run` from the `pa1/` directory, this path should point to `pa1/keywords.json`, which must be copied from the resources directory before the first run. The per-hit weights for primary and secondary keywords are controlled by `crawler.scoring.primaryWeight` and `crawler.scoring.secondaryWeight`, with recommended defaults of 0.18 and 0.09 respectively. `crawler.scoring.maxOccurrencesPerKeyword` caps how many times a single keyword can contribute to a URL's score, and has a significant effect on crawl behaviour: high values reward topic pages disproportionately, while lower values such as 3 produce a more balanced distribution across page types. The seed relevance score set in `crawler.scoring.seedRelevanceScore` must be strictly greater than the maximum possible keyword score achievable by any discovered URL, so that seed URLs always remain at the top of the frontier and are not displaced by highly scored discovered pages.

Concurrency is controlled by `crawler.nCrawlers`, which sets the maximum number of concurrent pipeline tasks and is also exposed as the `--n-crawlers` command line argument. This parameter acts as a global semaphore rather than creating independent pollers, and its value should be kept consistent with the database pool size and the headless browser session limit set in `crawler.fetch.maxHeadlessSessions`. Additional fetch parameters include `crawler.fetch.connectTimeoutMs`, `crawler.fetch.readTimeoutMs`, `crawler.fetch.maxRedirects`, and `crawler.fetch.headlessCircuitOpenThreshold`, which controls how many consecutive headless failures cause the headless path to be temporarily bypassed. Politeness timing is governed by `crawler.rateLimit.minDelayMs` and `crawler.rateLimit.maxBackoffMs`. Robots caching behaviour is controlled by `crawler.robots.cacheTtlHours`, `crawler.robots.cacheMaxEntries`, and `crawler.robots.temporaryDenyMaxMinutes`, which sets the ceiling on how long a temporarily denied domain will be held back before being retried. Frontier lease behaviour is tuned through `crawler.frontier.leaseSeconds` and `crawler.frontier.leaseRecoveryBatchSize`, and the observability heartbeat interval is set via `crawler.health.heartbeatIntervalMs`.

Before starting a crawl, the following checklist should be verified: PostgreSQL is running with schema version 7 applied and JDBC properties are correct; `pa1/keywords.json` is present at the path referenced by `crawler.scoring.keywordConfig`; `crawler.budget.maxTotalPages` and the seed list are aligned with the goals of the experiment; `crawler.scoring.seedRelevanceScore` is strictly greater than the maximum possible keyword score; and `./gradlew run` is invoked with `--n-crawlers N` where N is consistent with the database pool and headless session limits.

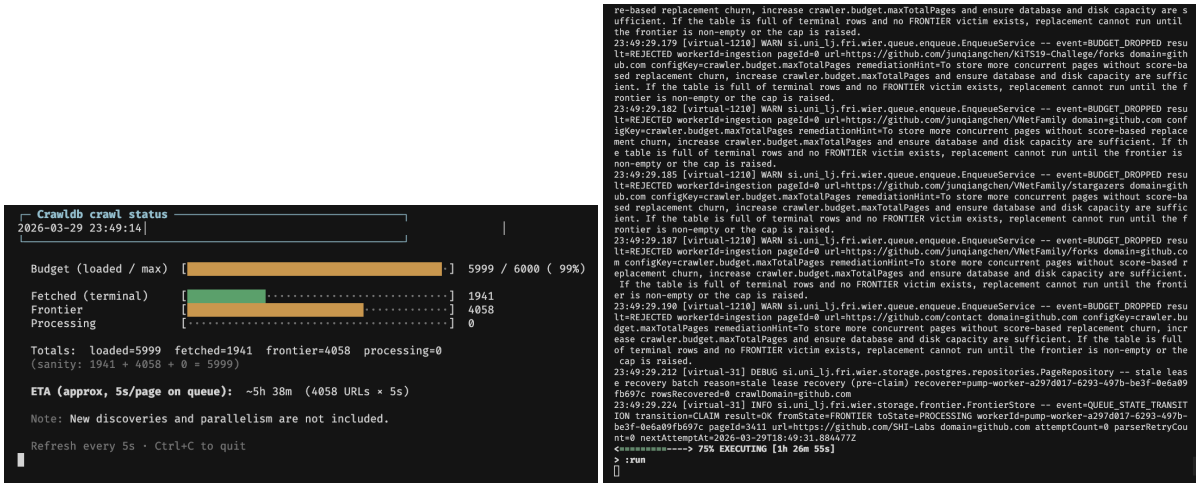
5 PROBLEMS AND SOLUTIONS

Several non-trivial problems were encountered during development, each of which required a targeted change to either the configuration, the scheduling logic, or the crawl workflow.

The first and most persistent problem was keyword over-breadth. The initial keyword list was large and general, covering broad machine learning and tooling vocabulary that appears widely across GitHub, including in site chrome such as pricing pages that mention Copilot or general ML features. This caused the relevance scorer to assign inflated scores to pages with no connection to image segmentation. The solution was to replace the broad keyword list with a shorter, segmentation-focused vocabulary, dramatically reducing score noise from unrelated pages.

Closely related was the problem of `maxOccurrencesPerKeyword` being set too high. With the cap at 128, topic pages dominated the frontier because their tag clouds and metadata caused every keyword to reach its maximum occurrence count, producing artificially high scores that no repository page could match. Lowering the cap to a value such as 3 rebalanced the scoring distribution, though it introduced a trade-off in that genuinely keyword-dense repository pages also received lower scores as a result.

Even after keyword tuning, the crawl continued to converge on topic pages rather than repositories. The root cause was



[H]

Figure 1. In-house tools used to monitor the crawler during production runs. Overall crawl progress was tracked through a dedicated status terminal program (left), while debugging was assisted by structured signals reported by internal function calls throughout the crawler pipeline (right).

structural: topic pages aggregate relevant keywords by design, so any occurrence-weighted scoring scheme will tend to favour them. The solution was the two-phase workflow described in Section 3. In the first phase, the occurrence cap was set high and the page budget was kept small, deliberately allowing the crawler to identify the most relevant topic pages. These were then added as seeds and topic page discovery was blocked for the second phase, which ran with the full page budget and converged on repository content.

The single-domain scheduling problem arose because `github.com` hosts the overwhelming majority of discovered URLs. When many workers each claimed the globally highest-scored URL from the frontier independently, they all converged on the same host simultaneously and stalled waiting for a politeness token, generating a large number of lease reschedules without producing useful fetches. The domain-scoped frontier pump was introduced to solve this: it serialises claims per domain, acquires the politeness token before handing work to a worker, and ensures that the first fetch hop does not consume a second token, keeping the worker pool productive without violating the rate limit.

Subdomain noise emerged early in testing, when the database accumulated many `*.github.com` hosts such as `api.github.com`, `cli.github.com`, and `help.github.com`, all carrying a relevance score of zero and containing no useful content. The crawl scope was tightened to admit only `github.com` and `www.github.com`, with `www` treated consistently with the apex domain for scope checks, site key assignment, and deduplication to avoid creating separate records for the same logical host. A related issue was that redirects between the `www` variant and the apex could cause robots alignment problems, which was resolved by normalising both to the same site key throughout the pipeline.

Near-duplicate URLs, such as `.../search` and its localised variant `.../search/en`, were handled through the canonical duplicate relationship in storage: the localised variant is recorded as a duplicate and linked to the canonical original without storing a second page body, preserving the link graph edge while avoiding redundant content.

The headless browser introduced cost disproportionate to its benefit on `github.com`, where most pages are server-rendered. The fetcher was configured to prefer the plain HTTP client and fall back to headless only when the HTTP response contained no links or included a JavaScript-disabled message, keeping headless session usage low for the majority of pages.

Finally, a score batching effect was observed in the database, where large groups of frontier rows shared the same relevance score. This was traced to the fact that page-level context signals such as title and metadata are applied uniformly to every outlink discovered from a given hub page, and substring scoring of URL paths produces identical scores for structurally similar URLs. This behaviour is expected given the scoring design and does not affect correctness, though it means that tie-breaking by insertion timestamp plays a larger role in frontier ordering than might be anticipated.

6 DATABASE STATISTICS

The crawl was restricted to the `github.com` domain. Prior to scope enforcement, the database accumulated a number of subdomains such as `api.github.com`, `cli.github.com`, and `help.github.com`, all of which carried a relevance score of

[H]

Table 2. Summary statistics of the production crawl database.

Metric	Value
Sites crawled	1
Total pages	5,000
HTML pages	5,000
Binary pages	0
Duplicate pages	0
Distinct content hashes	5,000
Total links	197,401
Total images	14,221
Pages with at least one image	3,079
Average images per page (all pages)	2.84
Average images per page (pages with images)	4.62

zero and contained no useful content. These were subsequently excluded via the domain allowlist. The absence of binary pages is a consequence of GitHub's rendering behaviour: file types such as `.py` and `.yaml` are served as HTML views with embedded content rather than as raw bytes, meaning that file URLs on `github.com` itself rarely resolve to true binary bodies. True binaries such as PDFs were encountered during testing but were hosted off-site and therefore fell outside the crawl scope. Additionally, many binary-adjacent paths on GitHub are disallowed by `robots.txt`, for example deep `blob/` paths, further limiting their presence in the dataset.

Duplicate pages were not observed in the production crawl, though test runs did surface cases such as `.../search` and its translated variant `.../search/en`, where the localised version was treated as a duplicate and linked canonically to the original. The 197,401 collected links reflect the densely interconnected nature of GitHub as a platform, where repositories, topics, user profiles, and issue trackers form a highly connected graph. The equality between total pages and distinct content hashes confirms that no two crawled pages shared identical content, consistent with the zero duplicate count. Finally, the 3,079 pages containing at least one image correspond predominantly to repository README pages, which typically include figures, diagrams, or badges as part of their project overview. Pages such as issue threads or individual documentation files rarely contain embedded images, which accounts for the relatively low fraction of image-bearing pages overall.

Figure 2 presents a visualisation of a representative subset of the crawled link graph, rendered with 700 nodes due to memory constraints imposed by the visualisation tooling. Despite covering only a fraction of the full 5,000-page dataset, the graph reveals clear structural patterns consistent with the nature of GitHub as a platform.

The most prominent feature is a set of dense, flower-like clusters distributed across the graph. Each such cluster originates from a central hub node corresponding to one of the seeded URLs, either a topic page or a well-known repository, from which a large number of outgoing links radiate outward to closely related pages. Within each cluster, nodes are heavily interconnected, reflecting the tendency of GitHub pages to cross-link to related repositories, contributors, and topics within the same neighbourhood of content.

These clusters are not isolated. Inter-cluster edges connect the dense groupings to one another, forming a larger connected component. This mirrors the broader structure of GitHub, where repositories sharing topics, authors, or dependencies naturally reference each other across different segments of the graph. The visualisation confirms that the preferential crawling strategy successfully captured a well-connected, thematically coherent subgraph centred on image segmentation content, rather than a fragmented or randomly dispersed collection of pages.

7 CONCLUSION

This report presented the design and implementation of a preferential web crawler targeting image segmentation content on GitHub. The crawler was built around a two-stage architecture that separates URL ingestion from page fetching, with a domain-scoped frontier pump ensuring that politeness constraints are respected without sacrificing worker throughput. Seed selection followed a deliberate two-step process that combined broad connectivity with manual curation, producing a seed list that anchored the crawl in thematically relevant content from the outset. Several non-trivial engineering challenges were encountered and resolved during development, including keyword score inflation, topic page dominance, single-domain scheduling contention, and subdomain noise, each of which required targeted changes to the configuration, the scheduling logic, or the crawl workflow. The production crawl collected 5,000 pages and over 197,000 links without encountering any duplicate pages, and the resulting link graph exhibits a

[H]

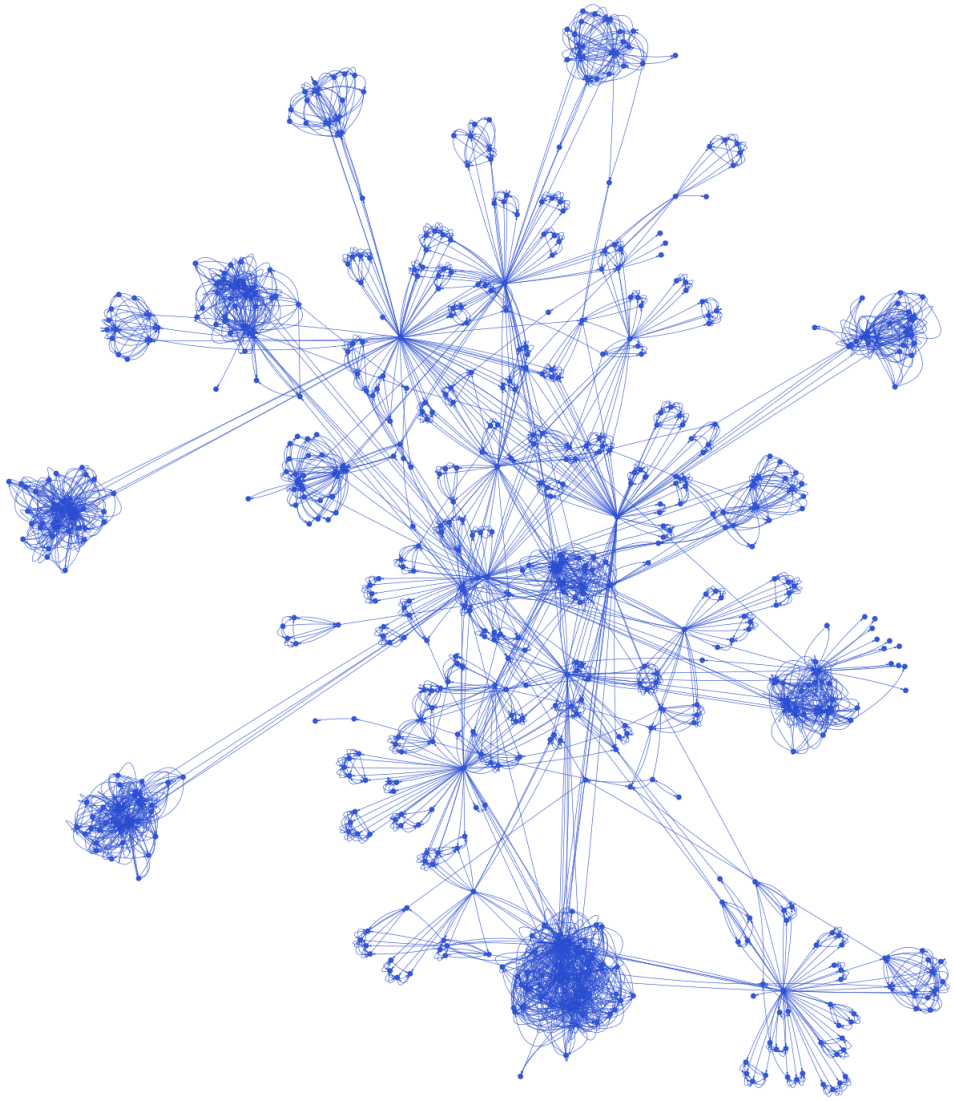


Figure 2. Visualisation of a representative subset of the crawled link graph, rendered with 700 nodes. Dense flower-like clusters originate from central hub nodes corresponding to seeded URLs, with heavy intra-cluster connectivity and inter-cluster edges forming a larger connected component consistent with the thematically coherent structure of the crawl.

well-connected cluster structure consistent with the domain focus. The dataset is well-suited for downstream retrieval-augmented generation applications, where the depth and coherence of the collected repository content is more valuable than breadth across loosely related pages.